

TP4 FoodStore - Reportes analíticos

Ejercicio de optimización de consultas con JOIN, agregación y ventana

Grupo: Franco Sampieri, Mauro Di Pietro, Renzo Victoria, Tomas Martinez, Alam Gonzalez

Repositorio del proyecto: https://github.com/francosampieri/TP2_BDD2_FoodStore.git

1. Consultas analíticas y reescrituras

Consulta de facturación por categoría y mes

```
detalle_pedido -> pedido -> producto -> categoria  
GROUP BY mes y categoria; SUM(dp.subtotal)
```

El plan original usó Parallel Hash Join entre detalle_pedido y pedido, y luego Hash Join con producto y categoria. El cuello de botella fue el Sort previo al GroupAggregate, que usó external merge y disco.

Resultado original: 946,611 ms.

Reescritura probada: agrupar por id_categoria y resolver el nombre de la categoria al final.

Resultado de la reescritura: 1.089,669 ms. Se descartó porque empeoró el tiempo real.

Consulta de ranking de usuarios por gasto

```
pedido -> detalle_pedido -> usuario  
COUNT(DISTINCT pedido), SUM(subtotal), RANK por gasto
```

El plan original combinó detalle_pedido con pedido y usuario mediante Hash Join. El ordenamiento previo al GroupAggregate procesó aproximadamente 400.000 filas y escribió datos a disco.

Resultado original: 1.283,410 ms.

Reescritura probada: preagregar los detalles por pedido antes de agrupar por usuario.

Resultado de la reescritura: 2.599,644 ms. Se descartó por generar agregaciones hash con múltiples batches y uso de disco.

2. Lectura crítica de un plan con JOIN

Se utilizó el plan original de facturación por categoría y mes. La explicación generada por IA identificó el Incremental Sort, la agregación parcial y final, los Hash Join y el Parallel Hash Join.

Afirmaciones aceptadas: 1 a 7 y 9. La explicación distinguió correctamente costo estimado de tiempo real, el uso de external merge y el rol de los workers.

Afirmación corregida: la última oración de la afirmación 8 indicó que existía un Hash simple sobre pedido.

El plan real no tiene un Hash simple sobre pedido. Pedido se lee mediante Parallel Seq Scan y se construye un Parallel Hash, utilizado por el Parallel Hash Join contra detalle_pedido. Los Hash simples del plan corresponden a producto y categoría.

3. Ranking y subconsulta correlacionada

Ranking con ventana: se generó un ranking de productos por facturación dentro de cada categoría, usando RANK() y filtros de baja lógica en categoría, producto y detalle_pedido.

Subconsulta correlacionada: se generaron pedidos cuyo total es mayor al promedio de pedidos no eliminados del mismo usuario.

Cada consulta tuvo una alternativa con estructura distinta y se verificó formalmente su equivalencia mediante EXCEPT.

```
A.1: consulta de ranking menos alternativa = 0 filas
A.2: alternativa menos consulta de ranking = 0 filas
B.1: correlacionada menos alternativa = 0 filas
B.2: alternativa menos correlacionada = 0 filas
```

Los resultados confirman que ambas versiones de cada consulta devolvieron el mismo conjunto de filas.

4. Competencia de optimización

Por indicación explícita del profesor, la competencia reutilizó la consulta común del TP3. Esta indicación específica prevalece sobre el ejemplo general de consulta analítica con JOIN y agregación de la guía.

```
SELECT id_producto, nombre_producto, precio, stock
FROM producto
WHERE id_categoria = 1 AND disponible = TRUE
      AND eliminado = FALSE AND precio BETWEEN 1000 AND 3000
ORDER BY precio DESC;
```

Estrategia aplicada: reutilización del índice parcial compuesto idx_producto_cat_precio_disp_vig.

Antes: 93,394 ms. El índice por categoría recuperaba 7.147 filas y luego descartaba aproximadamente 3.980.

Después: 10,855 ms. El índice parcial devolvió 3.167 filas sin Rows Removed by Filter.

Mejora: aproximadamente 8,6 veces.

El Sort se mantuvo en ambos planes. La mejora se produjo porque el índice parcial compuesto redujo el trabajo de filtrado antes de llegar al ordenamiento.

5. Uso de inteligencia artificial

Kiro se utilizó para redactar las especificaciones precisas de las consultas de la Parte 3. OpenCode se utilizó para seleccionar consultas, proponer reescrituras, explicar el plan y generar alternativas equivalentes. Cada propuesta se contrastó con planes o verificaciones reales antes de aceptarse o descartarse.

Conclusión

El trabajo permitió comprobar que una reescritura no garantiza una mejora: las dos alternativas analíticas se descartaron por empeorar el tiempo real. En cambio, el índice parcial reutilizado en la consulta común mantuvo una mejora significativa y ya validada sobre foodstore_tp3.